

Improved Proposed Wording for LWG 2114 (contextually convertible to bool)

- [Introduction](#)
- [Revision History](#)
- [Discussion](#)
- [Rationale](#)
- [Questions to Committee](#)
- [Exotic Types](#)
- [Proposed Resolution](#)
- [Resolved Issues](#)
- [Proposed Resolution](#)
- [Acknowledgements](#)
- [Bibliography](#)

Introduction

This proposal is intended to provide wording to resolve the existing library issue [LWG 2114](#).

Revision History

Changes since [P2167R1](#):

- Rebase wording to [N4910](#).
- Present four places with *mutually exclusive* wording alternatives (to be decided for) and add a new section [Questions to Committee](#) to ensure that the intended design is ensured and that possible wording alternatives have been considered and the non-selected ones are intentionally rejected.
- Add a new section [Exotic Types](#) that present intentionally exotic types whose behaviour would change depending on some solution choices.

Changes since [P2167R0](#):

- Remove changes suggested for [valarray.comparison]. For the same reason as e.g. for [optional.relops] and [reverse.iter.cmp] the result is just converted to bool and nothing else is done with it.
- Rebase wording to [N4892](#).
- Improve document title.

Discussion

Issue [LWG 2114](#) has already a long history and a number of wording revisions that went backward and forward. But with the introduction of the *boolean-testable* exposition-only concept by [P1964R2](#) adopted during the Prague 2020 meeting we have now an officially accepted useful tool that can be used to fix this issue.

Since the edits of the working draft are not very small, this paper has been written.

Rationale

The *boolean-testable* concept has all the properties that [LWG 2114](#) tried to specify without having language-concepts available. Our existing terminology of "modeling" a concept should allow us to use this wording even when we specify something within existing named requirements sets. So the approach of this paper is just to impose the *boolean-testable* concept requirements to those places that the issue identifies, now updated to the most recent working draft.

The following guidelines have been chosen to decide for applying the *boolean-testable* concept requirements below:

- Type families that are potentially influenced by program-defined types, such as types meeting the *Cpp17NullablePointer* requirements, are usually good candidates, unless the `bool`-like expressions are purposely intended to reflect the exact behaviour as presented (The design of the comparison functions of `optional` belong into this category)
- Type families or specifications that solely exist to shorten the specification of library-provided types, should usually *not* be good candidates, unless the intention exists here to provide Library implementors more freedom. An example for this case are the Container requirements which are actually invented to shorten the specification of the existing concrete container templates, where all of them use a concrete type `bool` for `operator==/!=` and for the `empty` function

Questions to Committee

The following lists a number of explicit questions the author would like to ask the LWG committee for its opinion to ensure that the paper meets its existing design *intention*. The paper doesn't intend to change the design intention, but attempts to clarify and possibly harmonize them, so LWG may need to temporarily forward some of these questions to LEWG.

[1.](#) Regarding 24.2.2.2 [container.reqmts] should for the container requirements of `operator==/!=` and `empty()` the return type `bool` be enforced? According to "gentlemen's agreement" the container requirements are actually considered as a simplifying means to specify concrete Standard library container requirements and are not actually representing more general applicable requirements (albeit some books such as *Generic Programming and the STL* by Matthew Austern and the [original SGI specification](#) represent this differently). Currently all library-provided containers (`valarray` is not such thing!) do have declared `empty` functions and `operator==/!=` that return `bool`, but we may want to support others in the future. This particular change would be not essential for this intended bug-fix paper and could be done by an independent proposal if really needed. Two possible counter arguments against such a change are also that (a) user code could potentially attempt to specialize the a Standard container template for a program-defined template argument type and this wording change could potentially break such a program-defined specialization and (b) this could be a potential unnecessary restriction for the forward evolution of the Standard Library (Consider a potential future container type that would like to deviate from a `bool` return type).

It should be pointed out that in the past we had considered to take advantage of "convertible to `bool`" requirements for allocator equality by allowing to implement them

as constexpr with std::true/false_type as return types, but with the introduction of the is_always_equal query type of an allocator we had withdrawn this idea, so at the moment we have no need for such a customization point. If we really want to return to a similar point again, we can easily decide to change the return type e.g. for container equality again to model *boolean-testable*, but now specifically to realize a concrete purpose. But this argument holds only for non-program-defined (specializations of) container types.

The affected wording location is [\[container.reqmts\]](#).

2. Independent from the previous bullet but based on similar arguments as this proposal uses for the currently suggested 24.2.2.2 [container.reqmts] wording changes, the wording changes to fpos (31.5.3.2 [fpos.operations] Table [tab:fpos.operations]) could similarly be strengthened to enforce a bool return type for operator== and operator!= the argument being that this type is under control of the implementation and all currently inspected library implementations (libc++, libstdc++, MSVC STL) do already implement this enforcement. A possible counter against this position could be that user code could potentially attempt to specialize the std::pos template for a program-defined state type and this wording change could potentially break such a program-defined specialization. The affected wording location is [\[fpos.operations\]](#).

3. Previous revisions of this proposal did only suggest to add *boolean-testable* constraints to tuple's operator==, but due to the strong type similarity between pair and tuple one could argue that pair's operator== should have a similar restriction. The committee should decide which of these positions is more important. Adding the *boolean-testable* could potentially invalidate existing program-defined types for pair. The affected wording locations are [\[pairs.spec\]](#) and [\[tuple.rel\]](#).

4. There exist a number of places in the standard wording (usually from its older parts), that could be modified as follows (The presented example is from 24.3.9.6 [forward.list.ops]):

```
size_type remove(const T& value);
template<class Predicate> size_type remove_if(Predicate pred);
```

-13- *Effects*: Erases all the elements in the list referred to by a list iterator *i* for which the following conditions hold: `bool(*i == value)` (for `remove()`), `bool(pred(*i))` is true (for `remove_if()`). Invalidates only the iterators and references to the erased elements.

[...]

```
size_type unique();
template<class BinaryPredicate> size_type unique(BinaryPredicate binary_pred);
```

[...]

-20- *Effects*: Erases all but the first element from every consecutive group of equivalent elements. That is, for a nonempty list, erases all elements referred to by the iterator *i* in the range `[begin() + 1, end())` for which `bool(binary_pred(*i, *(i - 1)))` is true. Invalidates only the iterators and references to the erased elements.

[...]

More modern wording usually uses such an explicit bool conversion in corresponding wording, see e.g. from 25.4.4.2 [range.iter.op.advance]:

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr void ranges::advance(I& i, S bound);
```

[...]

(4.3) — Otherwise, while `bool(i != bound)` is true, increments `i`.

This paper *could* do such additional wording clean-up (but hasn't done yet), but the number of affected places is rather large and the required wording effort could reduce the interest on this paper. The author is willing to do that work, if the committee prefers it, but doesn't this change would be a necessary part of this proposal.

Exotic Types

The following constructed example would potentially cause different behaviour depending on the selected wording for pair and tuple. This example is not intended of being useful, but exist for demonstration purposes only (Note that currently no semantic requirements are imposed on `operator==` for the presented types).

This example would be *ill-formed* by any of [\[pairs.spec\] Option B](#), [\[pairs.spec\] Option C](#), or [\[pairs.spec\] Option D](#).

```
#include <utility>
#include <tuple>
#include <iostream>

struct A { int val; };
struct B { int val; };
struct EqA { int val; explicit operator bool() const { return val > 0; } };
struct EqB { int val; explicit operator bool() const { return val == 0; } };
struct AndAB { int val; operator bool() const { return val < 0; } };

EqA operator==(A x, A y) { return EqA{x.val + y.val}; }
EqB operator==(B x, B y) { return EqB{x.val + y.val}; }
AndAB operator==(EqA x, EqB y) { return AndAB{x.val - y.val}; }

int main()
{
    std::pair<A, B> pab1(A{1}, B{2}), pab2(A{4}, B{6});

    // static_assert(!boolean-testable<decltype(pab1.first == pab2.first)>);
    // static_assert(!boolean-testable<decltype(pab1.second == pab2.second)>);

    std::cout << (pab1 == pab2) << std::endl;
    std::tuple<A, B> tab1(A{1}, B{2}), tab2(A{4}, B{6});
    std::cout << (tab1 == tab2) << std::endl;
}
```

Output according to existing wording:

```
1
0
```

Resolved Issues

If the proposed resolution will be accepted, the following library issues will be resolved:

Number	Description
LWG 2114	Incorrect " <i>contextually</i> convertible to <code>bool</code> " requirements

Proposed resolution

The proposed wording changes refer to [N4910](#).

1. Modify in 16.4.4.2 [utility.arg.requirements] Table [tab:cpp17.equalitycomparable] as indicated:

Table 27: *Cpp17EqualityComparable* requirements [tab:cpp17.equalitycomparable]

Expression	Return type	Requirement
<code>a == b</code>	convertible to bool <code>decltype(a == b)</code> . models <i>boolean-testable</i>	<code>==</code> is an equivalence relation, that is, it has the following properties: [...]

2. Modify in 16.4.4.2 [utility.arg.requirements] Table [tab:cpp17.less-than-comparable] as indicated:

Table 28: *Cpp17LessThanComparable* requirements [tab:cpp17.less-than-comparable]

Expression	Return type	Requirement
<code>a < b</code>	convertible to bool <code>decltype(a < b)</code> . models <i>boolean-testable</i>	<code><</code> is a strict weak ordering relation (27.8 [alg.sorting])

3. Modify in 16.4.4.4 [nullablepointer.requirements] Table [tab:cpp17.nullablepointer] as indicated:

Table 35: *Cpp17NullablePointer* requirements [tab:cpp17.nullablepointer]

Expression	Return type	Operational semantics
...		
<code>a != b</code>	contextually convertible to bool <code>decltype(a != b)</code> . models <i>boolean-testable</i>	<code>!(a == b)</code>
<code>a == np</code> <code>np == a</code>	contextually convertible to bool <code>decltype(a == np) and decltype(np == a)</code> . both model <i>boolean-testable</i>	<code>a == P()</code>
<code>a != np</code> <code>np != a</code>	contextually convertible to bool <code>decltype(a != np) and decltype(np != a)</code> . both model <i>boolean-testable</i>	<code>!(a == np)</code>

4. Modify 17.11.6 [cmp.alg] as indicated:

[...]

-4- The name `compare_strong_order_fallback` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Given subexpressions `E` and `F`, the expression `compare_strong_order_fallback(E, F)` is expression-equivalent (3.21 [defs.expression.equivalent]) to:

[...]

(4.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and ~~convertible to bool~~ `decltype(E == F)` and `decltype(E < F)` both model *boolean-testable*,

```

E == F ? strong_ordering::equal :
E < F  ? strong_ordering::less  :
        strong_ordering::greater

```

except that E and F are evaluated only once.

[...]

[...]

-5- The name `compare_weak_order_fallback` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Given subexpressions E and F, the expression `compare_weak_order_fallback(E, F)` is expression-equivalent (3.21 [defns.expression.equivalent]) to:

[...]

(5.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and convertible to `bool`, the types `decltype(E == F)` and `decltype(E < F)` both model `boolean-testable`,

```

E == F ? weak_ordering::equivalent :
E < F  ? weak_ordering::less      :
        weak_ordering::greater

```

except that E and F are evaluated only once.

[...]

[...]

-6- The name `compare_partial_order_fallback` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Given subexpressions E and F, the expression `compare_partial_order_fallback(E, F)` is expression-equivalent (3.21 [defns.expression.equivalent]) to:

[...]

(6.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and convertible to `bool`, the types `decltype(E == F)` and `decltype(E < F)` both model `boolean-testable`,

```

E == F ? partial_ordering::equivalent :
E < F  ? partial_ordering::less      :
E > F  ? partial_ordering::greater :
        partial_ordering::unordered

```

except that E and F are evaluated only once.

[...]

5. Keep 21.3.9 [meta.logical] unchanged: These logical type traits do already the boolean logic for you, and no further requirements are imposed.

6. [pairs.spec] Modify 22.3.3 [pairs.spec] as indicated:

[Drafting note: Four mutually exclusive options denoted by [Option A](#), [Option B](#), [Option C](#), and [Option D](#) are presented with increasing strictness, see also [Questions to Committee bullet 3](#).]

Option A: Perform no changes and keep supporting element types where no *individual* requirements are imposed on `p1.first == p2.first` or `p1.second == p2.second`.

```
template<class T1, class T2>
constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

-1- *Returns:* `x.first == y.first && x.second == y.second`.

Option B: Is in sync with the specification of `pair`'s `operator<=>` and harmonizes `operator==` with the behaviour of a defaulted one (11.10.2 [class.eq]). This solution harmonizes best with that of [\[tuple.rel\]](#) **Option A**.

```
template<class T1, class T2>
constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

-1- *ReturnsEffects:* Equivalent to: ~~`x.first == y.first && x.second == y.second`~~

```
if (bool b = (x.first == y.first); !b) return false;
return x.second == y.second;
```

Option C: Require that the individual expressions `p1.first == p2.first` and `p1.second == p2.second` model *boolean-testable*, harmonizing the corresponding requirements for `operator==` of `pair` and `tuple`. This solution harmonizes best with that of [\[tuple.rel\]](#) **Option B**.

```
template<class T1, class T2>
constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

-?- *Mandates:* `decltype(x.first == y.first)` models *boolean-testable* and `decltype(x.second == y.second)` models *boolean-testable*.

-1- *Returns:* `x.first == y.first && x.second == y.second`.

Option D: Combines both **Option B** and **Option C**, but enforces the same expression semantics as for `tuple` and is in sync with the specification of `pair`'s `operator<=>`. This solution also harmonizes `operator==` with the behaviour of a defaulted one (11.10.2 [class.eq]).

```
template<class T1, class T2>
constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

-?- *Mandates:* `decltype(x.first == y.first)` models *boolean-testable* and `decltype(x.second == y.second)` models *boolean-testable*.

-1- *ReturnsEffects:* Equivalent to: ~~`x.first == y.first && x.second == y.second`~~

```
if (bool b = (x.first == y.first); !b) return false;
return x.second == y.second;
```

7. [\[tuple.rel\]](#) Modify 22.4.8 [tuple.rel] as indicated:

[Drafting note: Two mutually exclusive options denoted by [Option A](#) and [Option B](#) are presented with increasing strictness, see also [Questions to Committee bullet 3](#).]

Option A: Harmonize wording with that of tuple's operator<=> and effectively don't change existing requirements. This solution harmonizes best with that of [\[pairs.spec\]](#) **Option B**.

```
template<class... TTypes, class... UTypes>
constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

-1- **Mandates:** ~~For all i , where $0 \leq i < \text{sizeof}...(TTypes)$, $\text{get}<i>(t) == \text{get}<i>(u)$ is a valid expression returning a type that is convertible to bool.~~ $\text{sizeof}...(TTypes)$ equals $\text{sizeof}...(UTypes)$.

-2- **Returns/Effects:** ~~true if $\text{get}<i>(t) == \text{get}<i>(u)$ for all i , otherwise false.~~ For any two zero-length tuples e and f , $e == f$ returns true. Performs an equality comparison between t and u . For any two zero-length tuples t and u , $t == u$ returns true. Otherwise, equivalent to:

```
if (bool b = (get<0>(t) == get<0>(u)); !b) return false;
return t_tail == u_tail;
```

where r_{tail} for some tuple r is a tuple containing all but the first element of r .

-3- **Remarks:** ~~The elementary comparisons are performed in order from the zeroth index upwards. No comparisons or element accesses are performed after the first equality comparison that evaluates to false.~~ [Note ?: The above definition does not require t_{tail} (or u_{tail}) to be constructed. It might not even be possible, as t and u are not required to be copy constructible. Also, all equality operator functions are short circuited; they do not perform element accesses beyond what is required to determine the result of the comparison. — end note]

Option B: Strengthen the "convertible to bool" requirement to *boolean-testable*. This solution harmonizes best with that of [\[pairs.spec\]](#) **Option C**.

```
template<class... TTypes, class... UTypes>
constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

-1- **Mandates:** For all i , where $0 \leq i < \text{sizeof}...(TTypes)$, $\text{get}<i>(t) == \text{get}<i>(u)$ is a valid expression ~~returning a type that is convertible to bool~~ and the type $\text{decltype}(\text{get}<i>(t) == \text{get}<i>(u))$ models *boolean-testable*. $\text{sizeof}...(TTypes)$ equals $\text{sizeof}...(UTypes)$.

[...]

8. Keep 22.5.6 [optional.relops] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
9. Keep 22.5.8 [optional.comp.with.t] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
10. Keep 22.6.6 [variant.relops] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.

11. Keep 22.8.3.2.5 [expected.un.eq] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
12. Keep 22.8.6.7 [expected.object.eq] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
13. Keep 22.8.7.7 [expected.void.eq] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
14. [\[container.reqmts\]](#) Modify 24.2.2.2 [container.reqmts] as indicated:

[Drafting note: Two mutually exclusive options denoted by [Option A](#) and [Option B](#) are presented, see also [Questions to Committee bullet 1.](#)]

Option A: Impose the relaxed freedom to use any type modeling *boolean-testable* for operator `==`/`!=` and `empty`.

[Drafting note: The *boolean-testable* requirements already impose that the negation of the equality result also has to model *boolean-testable*, so no additional restrictions to `a != b` are necessary.]

[...]

`a == b`

-39- *Preconditions:* `T` meets the *Cpp17EqualityComparable* requirements.

-40- *Result:* ~~Convertible to `bool`~~ `decltype(a == b)` models *boolean-testable*.

-41- *Returns:* `equal(a.begin(), a.end(), b.begin(), b.end())`

[...]

-43- *Remarks:* `==` is an equivalence relation.

`a != b`

-44- *Effects:* Equivalent to `!(a == b)`.

[...]

`a.empty()`

-59- *Result:* ~~Convertible to `bool`~~ `decltype(a.empty())` models *boolean-testable*.

-60- *Returns:* `a.begin() == a.end()`

-61- *Complexity:* Constant.

-62- *Remarks:* If the container is empty, then `bool(a.empty())` is true.

Option B: Enforce `bool` return type for operator `==`/`!=` and `empty`.

[...]

`a == b`

-39- *Preconditions*: `T` meets the *Cpp17EqualityComparable* requirements.

-40- *Result*: ~~Convertible to~~ `bool`.

-41- *Returns*: `equal(a.begin(), a.end(), b.begin(), b.end())`

[...]

-43- *Remarks*: `==` is an equivalence relation.

`a != b`

-44- *Effects*: Equivalent to `!(a == b)`.

[...]

`a.empty()`

-59- *Result*: ~~Convertible to~~ `bool`.

-60- *Returns*: `a.begin() == a.end()`

-61- *Complexity*: Constant.

-62- *Remarks*: If the container is empty, then `a.empty()` is true.

15. Keep 25.3.4.4 [iterator.concept.winc] unchanged: It seems to me that sufficient wording exists to exclude funny integer-class types, and they are all under control by the implementation.

16. Modify in 25.3.5 [iterator.cpp17] Table [tab:inputiterator] and Table [tab:randomaccessiterator] as indicated:

Table 83: *Cpp17InputIterator* requirements (in addition to *Cpp17Iterator*)
[tab:inputiterator]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
<code>a != b</code>	contextually convertible to <code>bool</code> <code>decltype(a != b)</code> <i>models <code>boolean-testable</code></i>	<code>!(a == b)</code>	[...]
...			

[...]

Table 87: *Cpp17RandomAccessIterator* requirements (in addition to *Cpp17BidirectionalIterator*) [tab:randomaccessiterator]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
...			
<code>a < b</code>	contextually convertible to <code>bool</code>	<i>Effects</i> : Equivalent to: return	[...]

	<code>decltype(a < b).</code> <code>models boolean-testable</code>	<code>b - a > 0;</code>	
<code>a > b</code>	contextually convertible to <code>bool</code> <code>decltype(a > b).</code> <code>models boolean-testable</code>	<code>b < a</code>	[...]
<code>a >= b</code>	contextually convertible to <code>bool</code> <code>decltype(a >= b).</code> <code>models boolean-testable</code>	<code>!(a < b)</code>	[...]
<code>a <= b</code>	contextually convertible to <code>bool</code> <code>decltype(a <= b).</code> <code>models boolean-testable</code>	<code>!(a > b)</code>	[...]

17. Keep 25.5.1.8 [reverse.iter.cmp] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
18. Keep 25.5.3.8 [move.iter.op.comp] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.
19. Modify 27.2 [algorithms.requirements] as indicated:

[Drafting note: The wording changes below also fix (a) unusual wording forms used ("should work") which are unclear in which sense they are imposing normative requirements and (b) the problem, that the current wording seems to allow that the predicate may mutate a call argument, if that is not a dereferenced iterator. Upon applying the new wording it became obvious that the both the previous and the new wording has the effect that currently algorithms such as `adjacent_find`, `search_n`, `unique`, and `unique_copy` are not correctly described (because they have no iterator argument named `first1`), which could give raise to a new library issue.]

-7- When not otherwise constrained, the Predicate parameter is used whenever an algorithm expects a function object (22.10 [function.objects]) that, when applied to the result of dereferencing the corresponding iterator, returns a value testable as true. ~~In other words, if an algorithm takes Predicate pred as its argument and first as its iterator argument with value type T, it should work correctly in the construct `pred(*first)` contextually converted to `bool` (7.3 [conv])~~ the expression `pred(*first)` shall be well-formed and the type `decltype(pred(*first))` shall model *boolean-testable* (18.5.2 [concept.booleantestable]). The function object pred shall not apply any non-constant function through ~~the dereferenced iterator~~ its argument. Given a glvalue u of type (possibly const) T that designates the same object as *first, `pred(u)` shall be a valid expression that is equal to `pred(*first)`.

-8- When not otherwise constrained, the BinaryPredicate parameter is used whenever an algorithm expects a function object that when applied to the result of dereferencing two corresponding iterators or to dereferencing an iterator and type T when T is part of the signature returns a value testable as true. ~~In other words, if an algorithm takes BinaryPredicate binary_pred as its argument and first1 and first2 as its iterator arguments with respective value types T1 and T2, it should work correctly in the construct `binary_pred(*first1, *first2)` contextually converted to `bool` (7.3 [conv])~~ the expression `binary_pred(*first1, *first2)` shall be well-formed and the type `decltype(binary_pred(*first1, *first2))` shall model *boolean-testable*. Unless

otherwise specified, BinaryPredicate always takes the first iterator's value_type as its first argument, that is, in those cases when T value is part of the signature, ~~it should work correctly in the construct binary_pred(*first1, value) contextually converted to bool (7.3 [conv])~~ the expression binary_pred(*first1, value) shall be well-formed and the type decltype(binary_pred(*first1, value)) shall model *boolean-testable*. binary_pred shall not apply any non-constant function through ~~the dereferenced iterators~~ any of its arguments. Given a glvalue u of type (possibly const) T1 that designates the same object as *first1, and a glvalue v of type (possibly const) T2 that designates the same object as *first2, binary_pred(u, *first2), binary_pred(*first1, v), and binary_pred(u, v) shall each be a valid expression that is equal to binary_pred(*first1, *first2), and binary_pred(u, value) shall be a valid expression that is equal to binary_pred(*first1, value).

20. Modify 27.8.1 [alg.sorting.general] as indicated:

[Drafting note: The existing wording inherits all the good wording from BinaryPredicate, that we fixed above, so there is only little to do but specifying the conversion to bool less strict, since we already know that it is a type that models *boolean-testable*]

-2- Compare is a function object type (22.10 [function.objects]) that meets the requirements for a template parameter named BinaryPredicate (27.2 [algorithms.requirements]). The return value of the function call operation applied to an object of type Compare, when contextually converted to bool (7.3 [conv]), yields true if the first argument of the call is less than the second, and false otherwise. Compare comp is used throughout for algorithms assuming an ordering relation.

21. Keep 28.6.3.2 [valarray.comparison] unchanged: These operations just evaluate what they get and say, and no further requirements are imposed.

22. [fpos.operations] Modify in 31.5.3.2 [fpos.operations] Table [tab:fpos.operations] as indicated:

[Drafting note: Two mutually exclusive options denoted by [Option A](#) and [Option B](#) are presented, see also [Questions to Committee bullet 2.](#)]

[Option A](#): Impose the relaxed library freedom to use any type modeling *boolean-testable* for operator==/!=.

Table 119: Position type requirements [tab:fpos.operations]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
...			
p != q	convertible to bool decltype(p != q) models <i>boolean-testable</i>	!(p == q)	
...			

[Option B](#): Enforce bool return type for operator==/!=.

[Drafting note: The properties of operator== except its more specific return type are all specified by *Cpp17EqualityComparable* (See 31.5.3.2 [fpos.operations] p1). The wording change suggested here does *not* attempt to resolve [LWG 3118](#).]

Table 119: Position type requirements [tab:fpos.operations]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
...			
<code>p == q</code>	<code>bool</code>		
<code>p != q</code>	<code>convertible to bool</code>	<code>!(p == q)</code>	
...			

23. Modify 33.2.1 [thread.req.paramname] as indicated:

[Drafting note: The following performs some minor drive-by fixes to fix minor wording issues that would e.g. exclude normal function pointers to be used as predicate. Note that we intentionally do *not* describe a potentially const lvalue `pred`, since there is nothing in the specification that would imply or require that.]

-1- Throughout this Clause, the names of template parameters are used to express type requirements. ~~If a template parameter is named `Predicate`, `operator()` applied to the template argument shall return a value that is convertible to `bool`~~`Predicate` is a function object type (22.10 [function.objects]). Let `pred` denote an lvalue of type `Predicate`. Then the expression `pred()` shall be well-formed and the type `decltype(pred())` shall model *boolean-testable* (18.5.2 [concept.booleantestable]). The return value of `pred()`, converted to `bool`, yields `true` if the corresponding test condition is satisfied, and `false` otherwise. [...]

Acknowledgements

Thanks to Barry Revzin, Tim Song, and Tomasz Kamiński for reviewing this proposal and providing helpful improvement suggestions.

Bibliography

[N4910] Thomas Köppe: "Working Draft, Standard for Programming Language C++", 2022
<https://wg21.link/n4910>

[P1964R2] Tim Song: "Wording for *boolean-testable*", 2020
<https://wg21.link/p1964r2>

[LWG2114] Daniel Krügler: "Incorrect "*contextually* convertible to `bool`" requirements", 2011
<https://wg21.link/lwg2114>

[Aus2001] Matthew H. Austern: "Generic Programming and the STL", 2001, Addison-Wesley